AIORI Hackathon PS-022

eBPF/XDP-Based Rate Limiting for DNS Query Floods

TC-164

Abstract

This report investigates eBPF and XDP (eXpress Data Path) for mitigating DNS query floods. The implementation combines DNS packet parsing with per-source and global token-bucket rate limiting, alongside an adaptive per-source state machine. Experiments on two CloudLab nodes evaluated single-source flooding, distributed flooding, and high-rate DNS traffic generation. Static per-source limiting discarded a large fraction of single-source flood traffic while preserving 100% legitimate DNS success in the tested scenario. The distributed experiment exposed limitations in the adaptive logic and showed that global limiting can also affect legitimate queries.

Contents

1 Introduction **3**

2 Experimental Environment **3**

2.1 CloudLab topology 3

2.2 DNS service and tools 3

3 System Architecture and Implementation **4**

3.1 Packet processing 4

3.2 Static per-source limiter 4

3.3 Global limiter 4

3.4 Adaptive limiter 4

4 Experimental Methodology **5**

5 Experimental Results **5**

5.1 Baseline without XDP 5

5.2 P1: Single-source flood with static XDP 6

5.3 P2: Distributed flood with adaptive XDP 6

5.4 High-rate Python traffic generation 7

5.5 Consolidated results 8

6 Limitations **8**

6.1 Adaptive detection does not account for global drops 8
6.2 Global limiting can affect legitimate traffic 8
6.3 Insufficient baseline CPU pressure 8
6.4 Traffic generator limitations 9
6.5 CPU exhaustion was not demonstrated 9
6.6 Limited latency data, attack diversity, and repetitions 9

7 Discussion **9**

8 Future Improvements **9**

9 Conclusion **10**

1 Introduction

DNS is essential to network communication and can be targeted by denial-of-service attacks that generate large volumes of queries to consume server resources. XDP allows eBPF programs to inspect packets early in the Linux networking path and potentially discard unwanted traffic before it proceeds further through the stack.

The objectives of this project are to:

1. Identify DNS query packets using an eBPF/XDP program.
2. Implement per-source and global rate limiting.
3. Investigate adaptive rate limiting.
4. Measure the effect of flooding on legitimate DNS queries.
5. Compare observations with and without XDP protection.
6. Identify implementation and experimental limitations.

2 Experimental Environment

2.1 CloudLab topology

The experiment used two CloudLab nodes connected through a private experimental network.

Component	Node0	Node1
Role	Traffic generator and DNS client	DNS server and XDP filtering
Experimental IP	10.10.1.1	10.10.1.2
Experimental interface	`enp65s0f0np0`	`enp65s0f0np0`
Management interface	`eno33np0`	`eno33np0`

Table 1: CloudLab experimental topology

Node1's management interface used address `128.110.219.51/21`. Its experimental interface used `10.10.1.2/24`. The legitimate client used source address `10.10.1.1`. Flood source addresses were `10.10.1.101` for single-source tests and `10.10.1.101`–`10.10.1.125` for distributed tests.

2.2 DNS service and tools

The DNS service used `dnsmasq` on node1 at `10.10.1.2`. The test domain was `test.lab.test`, intended to resolve to `192.0.2.10`.

The software and tools included Linux, eBPF, XDP, libbpf, C, Python, `dnsmasq`, `dnsperf`, `sar`, `mpstat`, `pidstat`, and `dig`. The XDP program was attached in generic/SKB mode using `XDP_FLAGS_SKB_MODE`.

3 System Architecture and Implementation

3.1 Packet processing

The eBPF program, `dns_tracer.bpf.c`, processes incoming Ethernet frames and identifies IPv4 UDP packets directed to destination port 53. The processing sequence is:

1. Parse the Ethernet header and identify IPv4.
2. Identify UDP packets and check destination port 53.
3. Validate the DNS query.
4. Apply per-source rate limiting, followed by global rate limiting.
5. Pass or drop the packet.

Packets that do not match the DNS traffic criteria do not enter the DNS rate-limiting path. The userspace controller, `dns_tracer.c`, loads and attaches the program and configures its parameters.

3.2 Static per-source limiter

A token bucket is maintained for each source IP using an LRU hash map.

Parameter	Value
Per-source rate	1,000 packets/s
Per-source burst	100 packets

Table 2: Per-source token-bucket configuration

3.3 Global limiter

A separate global token bucket limits aggregate DNS traffic. Its shared map value is protected by a BPF spin lock.

Parameter	Value
Global rate	20,000 packets/s
Global burst	1,000 packets

Table 3: Global token-bucket configuration

The global limiter is applied after the per-source limiter.

3.4 Adaptive limiter

The adaptive mechanism classifies individual sources using an exponentially weighted moving average (EWMA) of their per-source drop ratio. Its states are `NORMAL`, `THROTTLED`, `BLOCKED`, and `PROBATION`.

Parameter	Value
Epoch duration	1,000 ms
EWMA alpha	50%
Throttle threshold	30%
Recovery threshold	10%
Maximum throttle level	4
Decay shift	1
Probation rate	10 packets/s
Clean probation epochs required	3
Initial block duration	5,000 ms
Maximum block duration	60,000 ms

Table 4: Adaptive rate-limiting configuration

Adaptive mode is disabled by default and enabled through the controller's `-adaptive` option. A key limitation is that drops caused by the global limiter are not included in the per-source adaptive score.

4 Experimental Methodology

Experiments covered baseline and XDP-protected scenarios. The legitimate client sent queries from `10.10.1.1` to `10.10.1.2`. The measurement program recorded queries sent, `NOERROR` responses, other response codes, timeouts, success percentage, and the number of flood packets sent.

Legitimate DNS success was calculated as:

$$Successrate = \frac{NOERRORresponses}{legitimatequeriessent} \times 100\%.$$

The principal scenarios were:

- **P1: Single-source flood** from one source.
- **P2: Distributed flood** from 25 source addresses.
- **P3: Periodic flood** with quiet and flood intervals.
- **High-rate generation** using Python and `dnsperf`.

5 Experimental Results

5.1 Baseline without XDP

Baseline P1, P2, and P3 tests reported 100% legitimate DNS success at 50 queries per second (QPS). CPU idle time was approximately 99.97%, corresponding to approximately 0.03% utilization in the reported observation. This workload established normal DNS operation but did not demonstrate significant CPU pressure or server resource exhaustion.

5.2 P1: Single-source flood with static XDP

The flood generator was configured for approximately 12,000 packets/s. XDP was attached to `enp65s0f0np0`, with a per-source rate of 1,000 packets/s and burst of 100, a global rate of 20,000 packets/s and burst of 1,000, and adaptive mode disabled. The legitimate client generated 500 QPS for 40 seconds.

Legitimate DNS metric	Result
Queries sent	20,000
NOERROR responses	20,000
Other response codes	0
Timed-out queries	0
Success rate	100%
Flood packets reported sent	479,924

Table 5: P1 legitimate DNS results

XDP counter	Value
Total packets	499,927
DNS packets passed	60,099
Malformed packets dropped	0
Per-source limit drops	439,825
Global limit drops	0
Blocked-source drops	0

Table 6: P1 XDP statistics

The per-source limiter discarded approximately 88% of the packets counted in this test, while the legitimate client maintained 100% success at 500 QPS. The global limiter was not triggered. These results support the effectiveness of static per-source limiting for the tested single-source workload, but do not establish how it would behave when legitimate and malicious traffic share a source or when traffic is distributed across many sources.

5.3 P2: Distributed flood with adaptive XDP

The distributed test used 25 source IP addresses, `10.10.1.101` through `10.10.1.125`, with a configured rate of approximately 900 packets/s per source. The theoretical aggregate flood rate was:

$$25 \times 900 = 22{,}500 packets/s.$$

The legitimate client continued to send 500 QPS.

Each source remained below the per-source threshold, so no per-source drops occurred and the adaptive state machine did not transition any source into throttled or blocked states. Aggregate traffic exceeded the global limit, producing 118,987 global-limit drops. Because the global limiter did not distinguish legitimate requests from flood traffic, legitimate DNS success fell to 80.38%. This exposes a limitation of the current design for distributed floods.

XDP metric	Result
Total packets	920,039
DNS packets passed	801,008
Per-source limit drops	0
Global limit drops	118,987
Malformed packet drops	0
Blocked-source drops	0
Entered throttled state	0
Entered blocked state	0
Entered probation state	0
Entered normal state	0

Table 7: P2 XDP statistics

Legitimate DNS metric	Result
Queries sent	20,000
NOERROR responses	16,075
Other response codes	0
Timed-out queries	3,925
Success rate	80.38%
Flood packets reported sent	Approximately 899,995

Table 8: P2 legitimate DNS results

5.4 High-rate Python traffic generation

A Python distributed generator used 25 source addresses and multiple threads. A configuration requested 9,000,000 packets/s per source, or a theoretical aggregate of 225,000,000 packets/s. The observed packet count was substantially lower.

Metric	Result
Flood packets reported sent	1,962,955
Test duration	40 s
Average generated rate	Approximately 49,074 packets/s
Observed receiver rate	Approximately 49,000–50,000 packets/s
Observed transmit rate	Approximately 39,000–40,000 packets/s
Reported interface utilization	Approximately 0.12%

Table 9: High-rate Python generator observations

The generator did not achieve its requested theoretical rate. Agreement between its packet count and the receiver's observed rate indicates that approximately 49,000–50,000 packets/s reached the experimental interface. These results do not establish the maximum capacity of the network or XDP program, and the low interface utilization does not indicate link saturation.

5.5 Consolidated results

Experiment	Traffic pattern	Legitimate QPS	Legitimate success	Main observation
Baseline P1	Single source, no XDP	50	100%	Normal DNS operation
Baseline P2	Distributed, no XDP	50	100%	Normal DNS operation
Baseline P3	Periodic, no XDP	50	100%	Normal DNS operation
XDP P1	Single source, static limiter	500	100%	439,825 source-limit drops
XDP P2	25 distributed sources, adaptive enabled	500	80.38%	Global drops; no adaptive transitions
Python high-rate test	Distributed continuous	500	100% reported	Approximately 49,074 flood packets/s

Table 10: Summary of principal experimental observations

These experiments did not use equivalent traffic rates and configurations, so the results should not be interpreted as a direct controlled comparison.

6 Limitations

6.1 Adaptive detection does not account for global drops

The adaptive score uses per-source drops but excludes global-limit drops. During the distributed test, each source remained below its individual threshold, so the adaptive state machine did not identify any source as suspicious. A separate global adaptive state machine or carefully designed aggregate signals could help address this limitation.

6.2 Global limiting can affect legitimate traffic

The global token bucket aggregates all matching DNS traffic and does not distinguish legitimate requests from flood traffic. When the aggregate rate exceeded the configured threshold, legitimate queries were also dropped, contributing to the 80.38% success rate in P2.

6.3 Insufficient baseline CPU pressure

Baseline CPU idle time was approximately 99.97%, and legitimate DNS success was 100% at 50 QPS. The baseline did not demonstrate CPU exhaustion or substantial DNS service degradation. The static limiter demonstrated packet filtering, but a reduction in CPU utilization under equivalent server-saturating workloads was not measured.

6.4 Traffic generator limitations

The Python generator's configured rate substantially exceeded its actual rate. The configuration requesting 225 million packets/s resulted in approximately 49,074 packets/s based on the reported packet count and duration. `dnsperf` generated more traffic, but its `REFUSED` responses exposed a separate DNS configuration or workload issue.

6.5 CPU exhaustion was not demonstrated

The available results do not establish that `dnsmasq` or the node1 CPU reached saturation. A stronger evaluation should simultaneously measure overall and per-core CPU utilization, `dnsmasq` CPU consumption, XDP processing overhead, DNS success, response latency, and packet-drop counts.

6.6 Limited latency data, attack diversity, and repetitions

The legitimate-query program recorded response codes and timeouts but not latency distributions. The tests focused primarily on UDP DNS traffic to port 53 and did not comprehensively evaluate other DNS patterns or source-address spoofing. The recorded results represent individual runs; repeated trials, averages, and confidence intervals were not established.

7 Discussion

The single-source experiment was the clearest successful result: the static per-source token bucket discarded 439,825 packets while the legitimate client maintained 100% DNS success at 500 QPS. The distributed experiment revealed a weakness in the adaptive design: individual sources stayed below their thresholds, so the adaptive mechanism did not activate. The global limiter dropped excess aggregate traffic but also affected legitimate requests.

These findings show that per-source and global limiting address different aspects of flooding, and neither mechanism alone provides complete protection against distributed attacks. The `dnsperf` experiment showed that a dedicated benchmarking tool generated substantially more traffic than the Python generator, but all completed responses were `REFUSED`; it therefore did not provide a valid measurement of successful DNS resolution under that workload.

Overall, the implementation is a functional prototype of an XDP-based DNS rate limiter. Further work is required before claiming complete protection against distributed DNS attacks or prevention of DNS server CPU exhaustion.

8 Future Improvements

1. **Native XDP support:** Investigate driver-mode attachment where supported.
2. **Global adaptive detection:** Add a global adaptive mechanism responsive to aggregate traffic pressure.
3. **Legitimate traffic protection:** Improve rate-limiting design so attack traffic does not consume capacity needed for legitimate requests.
4. **Improved traffic generation:** Use benchmarking or packet-generation tools with measured, repeatable rates.

5. **CPU measurements:** Collect per-core utilization and `dnsmasq` CPU consumption during all tests.
6. **Latency measurements:** Record median, p95, and p99 DNS response latency.
7. **DNS configuration validation:** Verify that the test domain returns `NOERROR` before each run.
8. **Repeated trials:** Run each scenario multiple times and report averages and confidence intervals.
9. **Additional attack patterns:** Vary source counts, packet rates, burst sizes, and traffic distributions.
10. **Adaptive threshold evaluation:** Evaluate EWMA thresholds and epoch durations for their effects on detection and legitimate traffic.

9 Conclusion

This project implemented and evaluated an eBPF/XDP-based DNS rate-limiting system in CloudLab. The implementation included DNS packet parsing, per-source token buckets, a global token bucket, and an adaptive per-source state machine.

The single-source experiment demonstrated effective packet filtering: with 479,924 flood packets reported by the generator, XDP recorded 439,825 per-source drops while all 20,000 legitimate DNS queries received `NOERROR`. The distributed experiment exposed a limitation of the adaptive mechanism: 25 flood sources remained below individual thresholds, no adaptive state transitions occurred, and legitimate DNS success decreased to 80.38% under global limiting.

The high-rate experiments showed that dedicated DNS benchmarking software generated substantially more traffic than the Python generator. However, the `dnsperf` test returned `REFUSED` for every completed query, so it cannot serve as evidence of successful DNS resolution under high load.

The principal conclusion is that **per-source XDP rate limiting can mitigate single-source DNS flooding, but the current adaptive and global limiting design needs further improvement to handle distributed floods without harming legitimate traffic.**